

Pure Python

Types

```
a = 2          # integer
b = 5.0        # float
c = 8.3e5      # exponential
d = 1.5 + 0.5j # complex
e = 4 > 5      # boolean
f = 'word'     # string
```

Lists

```
a = ['red', 'blue', 'green'] # manually initialization
b = list(range(5))           # initialize from
iteratable                   # iterable
c = [nu**2 for nu in b]      # list comprehension
d = [nu**2 for nu in b if nu < 3] # conditioned list
comprehension                # comprehension
e = c[0]                     # access element
f = c[1:2]                   # access a slice of the
list                         # list
g = c[-1]                   # access last element
h = ['re', 'bl'] + ['gr']   # list concatenation
i = ['re'] * 5               # repeat a list
['re', 'bl'].index('re')    # returns index of 're'
a.append('yellow')          # add new element to end
of list                     # of list
a.extend(b)                 # add elements from list
`b` to end of list `a`      # `b` to end of list `a`
a.insert(1, 'yellow')        # insert element in
specified position           # specified position
're' in ['re', 'bl']         # true if 're' in list
'fi' not in ['re', 'bl']     # true if 'fi' not in
list                         # list
sorted([3, 2, 1])            # returns sorted list
```

```
a.pop(2)          # remove and return item
at index (default last)
```

Dictionaries

```
a = {'red': 'rouge', 'blue': 'bleu'} # dictionary
b = a['red']                          # translate
item                                 # item
'red' in a                           # true if
dictionary a contains key 'red'      # dictionary a contains key 'red'
c = [value for key, value in a.items()] # loop through
contents                             # contents
d = a.get('yellow', 'no translation found') # return
default                             # default
a.setdefault('extra', []).append('cyan') # init key with
default                             # default
a.update({'green': 'vert', 'brown': 'brun'}) # update
dictionary by data from another one  # dictionary by data from another one
a.keys()                             # get list of
keys                                 # keys
a.values()                           # get list of
values                              # values
a.items()                            # get list of
key-value pairs                     # key-value pairs
del a['red']                          # delete key
and associated with it value         # and associated with it value
a.pop('blue')                        # remove
specified key and return the corresponding value
```

Sets

```
a = {1, 2, 3} # initialize
manually     # manually
b = set(range(5)) # initialize
from iterable  # from iterable
```

Strings

```
a = 'red'                # assignment
char = a[2]              # access individual
                           characters
'red ' + 'blue'          # string concatenation
'1, 2, three'.split(',') # split string into list
''.join(['1', '2', 'three']) # concatenate list into
string
```

Operators

```
a = 2                    # assignment
a += 1 (*=, /=)          # change and assign
3 + 2                    # addition
3 / 2                    # integer (python2) or float (python3)
division
3 // 2                   # integer division
3 * 2                    # multiplication
3 ** 2                   # exponent
3 % 2                    # remainder
abs(a)                   # absolute value
1 == 1                   # equal
2 > 1                    # larger
2 < 1                    # smaller
1 != 2                   # not equal
1 != 2 and 2 < 3         # logical AND
1 != 2 or 2 < 3          # logical OR
not 1 == 2               # logical NOT
'a' in b                 # test if a is in b
```

Control Flow

```
# if/elif/else
a, b = 1, 2
if a + b == 3:
    print('True')
```

```
elif a + b == 1:
    print('False')
else:
    print('?')

# for
a = ['red', 'blue', 'green']
for color in a:
    print(color)

# while
number = 1
while number < 10:
    print(number)
    number += 1

continue # continue to next iteration of loop
break # exit loop
```

Functions, Classes

```
# Function groups code statements and possibly
# returns a derived value
def myfunc(a1, a2):
    return a1 + a2

x = myfunc(a1, a2)

# Class groups attributes (data)
# and associated methods (functions)
class Point(object):
    def __init__(self, x):
        self.x = x
    def __call__(self):
        print(self.x)

x = Point(3)
```